

Adaptation du jeu Turing Machine au contexte de l'UTBM

Le jeu Turing Machine est conçu comme un jeu de déduction basé sur des règles avec lesquels un joueur doit déterminer un code bien précis. Dans notre projet, on a décidé d'adapter ce jeu au monde de l'UTBM.

Il s'agit de se mettre à la place de l'équipe administrative de l'UTBM chargée de l'attribution des salles de cours. En effet, le joueur doit résoudre des problèmes liés à l'attribution des salles en respectant des critères spécifiques mises en place par le jeu.

Le format du code qu'on a choisi d'utiliser prend la structure suivante :

- Campus : B pour Belfort, S pour Sevenans, M pour Montbéliard
- Bâtiment : pour compliquer un peu le jeu on a choisi de donner un même nom aux bâtiments de trois campus, sinon une résolution de la lettre correspondante au bâtiment enlèvera beaucoup de doutes sur la lettre du campus. Le bâtiment est représenté par une des lettres de A à E (5 possibilités)
- Etage : de 1 à 4
- Salle : de 0 à 9

On aura donc par exemple des solutions de la forme « BD25 » qui va désigner une salle au campus de Belfort, au bâtiments D, la 5^{ème} salle du 2^{ème} étage.

Les défis proposés incluent des situations réelles : l'attribution d'une salle d'examen à une matière d'humanité, l'organisation d'un TP de rattrapage ou encore la réservation d'une salle pour une conférence.

Les cartes critères présentes dans le jeu Turing Machine seront représentées dans notre jeu de la manière suivante : "Les bâtiments A, B et C sont occupés" ou "L'examen est prévu à un étage supérieur à 2". Ces critères vont aider le joueur à éliminer progressivement quelques possibilités et le poussent à déduire la bonne solution à ce problème

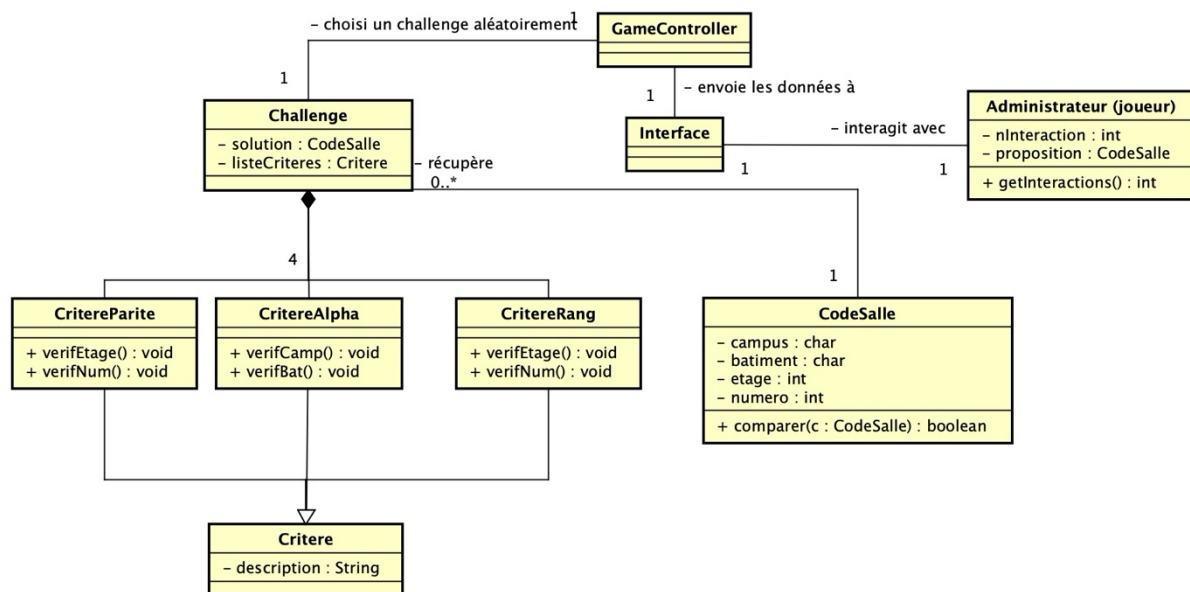
Fonctionnalités

Les fonctionnalités que nous avons imaginées sont les suivantes :

- Choix du challenge (parmi 3 ou 4)
- Une interface graphique avec une « fiche de notes » pour que le joueur puisse raisonner et organiser ses réflexions
- Un menu
- Des paramètres de difficultés (2, 3 ou 4 cartes critères)

Et sûrement d'autres choses que nous ajouterons au fil de l'avancée du code

Diagramme de classe

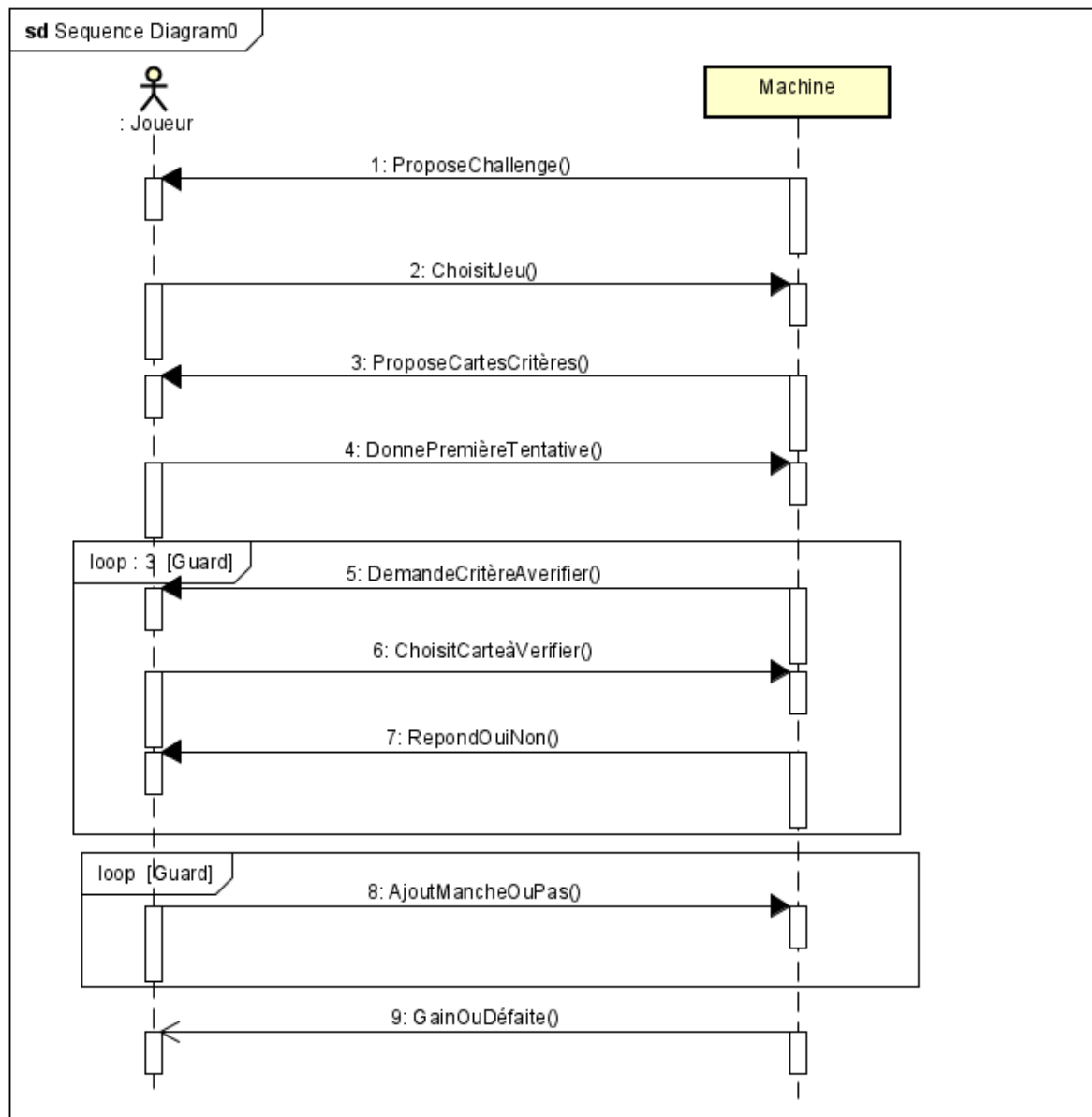


Les classes

Nous avons imaginé 6 classes principales pour le jeu de *Turing Machine*.

- 1) Tout d'abord, commençons par la base : le but du jeu étant d'établir la salle dans laquelle un cours ou un événement aura lieu, il faut donc une classe pour ces codes de salles : **CodeSalle**. Nous avons au début pensé à utiliser une structure de données basique (une struct) mais nous nous sommes rendus compte qu'une méthode pour comparer les codes était nécessaire.
- 2) Ensuite, nous avons imaginé la classe de la carte critère. Le problème avec celle-ci était la diversité des tests : il fallait séparer les différents types de critères et de vérificateurs. C'est pour cela que nous avons opté pour la création de 3 nouvelles sous-classes (peut-être plus dans le futur) :
- 3) **CritereParite** : permet de faire des tests sur la parité du digit de l'étage et du numéro de la salle
CritereAlpha : permet de faire des comparaisons basées sur l'ordre alphabétique du campus et du bâtiment
CritereRang : permet de faire des tests de comparaison sur les digits de l'étage et du numéro de la salle
- 4) **Challenge** correspond à une liste de 4 critères (peu importe quel type de critère), ainsi qu'un attribut solution qui n'est qu'un simple code de salle. Il s'agit de la classe qui représente un « problème » ou une « partie ».
- 5) La classe représentant le joueur est la classe **Administrateur**. Il s'agit pour l'instant d'une classe très simple, puisqu'elle n'est véritablement composée que de deux attributs : le nombre d'interactions restantes par tour et le code proposé à chaque tour.
- 6) Finalement, nous avons deux classes **GameController** et **Interface** qui n'ont pas encore été réellement développées car elles nécessitent de commencer dans la partie développement pour réellement comprendre comment les utiliser. La première est celle qui régit le backend du projet, et la deuxième le GUI.

Diagramme de Séquence :



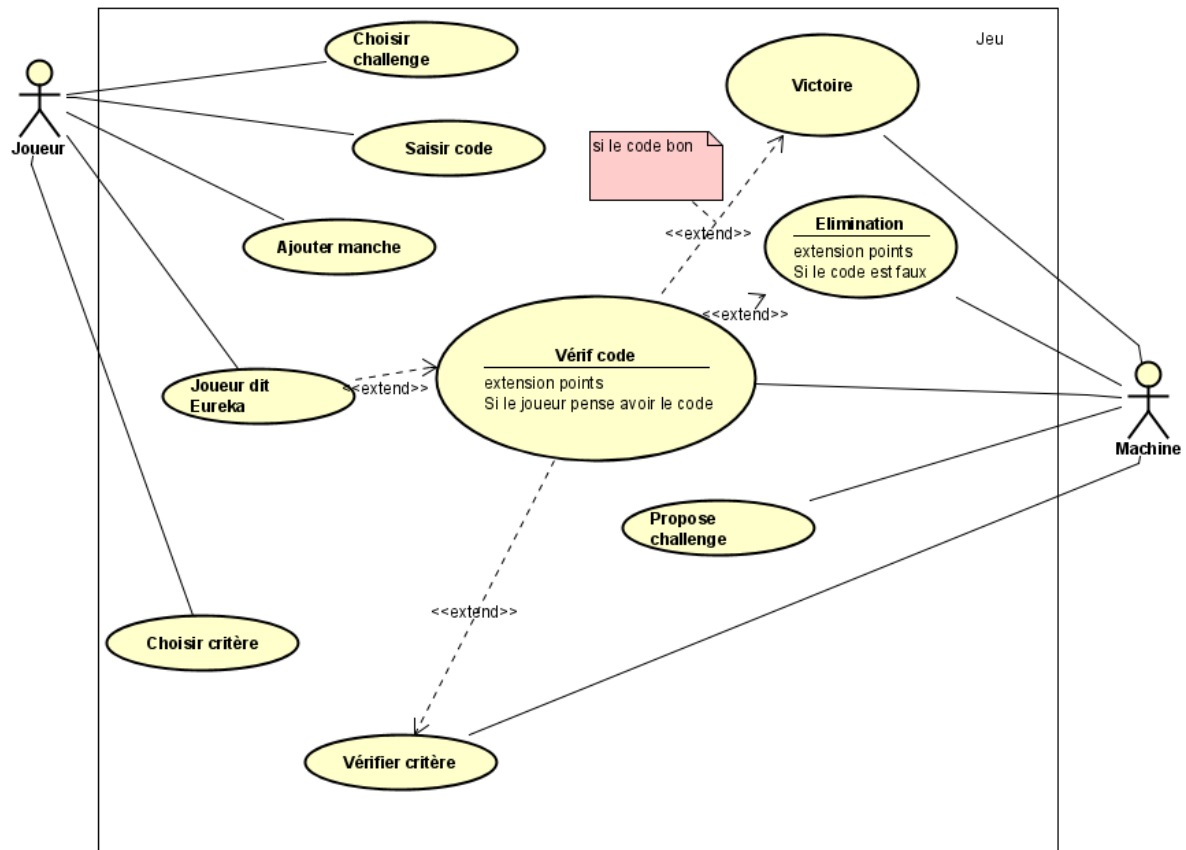
Ce diagramme de séquence propose l'ensemble des interactions qui peuvent avoir place entre la machine et le joueur dans l'ordre. En effet à l'ouverture du jeu, la machine propose au joueur de choisir un challenge parmi les 3 créés de notre part, chacun représentant un problème différent. Le joueur sélectionne le challenge qu'il souhaite jouer, et la machine lui fournit les cartes critères correspondantes.

Le joueur doit effectuer une première tentative en proposant un code. Ensuite la machine va demander au joueur quel critère il souhaite vérifier, selon la réponse du joueur elle va lui répondre par oui ou par non si ce critère a été vérifié dans le code qu'il a donné. Cette dernière étape peut être répétée jusqu'à 3 fois dans une seule manche.

A la fin le joueur va choisir s'il souhaite jouer une autre manche s'il n'a pas trouvé la solution ou de s'arrêter s'il pense l'avoir devinée.

à la fin du jeu et suivant le code final donné par le joueur, la machine annonce s'il a gagné ou perdu.

Diagramme de Cas d'Utilisation :



Le diagramme de cas d'utilisation représente les principales fonctionnalités du jeu et les interactions entre les acteurs (Dans notre cas : Joueur et Machine).

Le joueur commence par choisir un des challenges proposés par la machine, qui inclut des cartes critères spécifiques pour guider le joueur à trouver la solution. Une fois un code saisi, le joueur peut demander une vérification ou, s'il pense avoir trouvé la solution, déclarer "Eureka". Selon le résultat de la vérification de la machine, le joueur peut soit gagner, soit être éliminé.

S'il décide de ne pas déclarer "Eureka", une nouvelle manche commence entraînant une prolongation du jeu et une répétition des étapes précédentes jusqu'à ce que le joueur déclare "Eureka" et décide de mettre fin au jeu.

On a également mis le vérifier critère comme extension vue que le vérificateur du critère vérifie en quelque sorte une partie du code en comparant le code du joueur à la solution tout comme le vérificateur final du code.

Organisation des tâches

Pour l'instant, nous avons partagé les tâches d'une manière assez vague. Ali se charge du backend du projet et Mourad de l'interface graphique. Cela reste théorique car nous avons besoin de mieux avancer dans le projet pour pouvoir se donner précisément les tâches.

Nous utiliserons VSCode pour la programmation, GitHub pour la collaboration et la sauvegarde, et divers éditeurs de texte tels que Word ou Obsidian pour les rapports et des outils de communication pour rester en contact pendant les vacances de décembre.